



University of Engineering & Technology Peshawar

Computer Programming (C++) Lab

Lab 14

STUDENT NAME	Muhammad Sahal Ayub
REGISTRATION NO:	4042F24
COUSRE:	EE-170L COMPUTER PROGRAMMING LAB
DATE:	17/05/26
Lab Title:	Open Ended Lab (OEL) Grading System

SUBMITTED TO ENGR RIZWAN ULLAH

Lab 14(OEL)

Opening Ended Lab

Grading system

1. Objective:

The objective of this lab project is to design and implement a Relative Grading System in C++ that manages student records across multiple papers/subjects. The system assigns grades based on the relative performance of students within the same paper, computes per-student GPA and CGPA, and generates a Detailed Marks Certificate (DMC) transcript. The project demonstrates the use of structures, arrays, functions, sorting, and input validation in C++.

2. Tools & Environment:

Tool / Component	Details
Programming Language	C++ (Standard: C++11 or later)
Compiler	g++ / MinGW / MSVC
IDE	Visual Studio Code / Code::Blocks / Dev-C++
Libraries Used	<iostream>, <iomanip>, <cmath>, <string>
Platform	Windows / Linux / macOS

3. Theory & Background:

3.1 Relative Grading

Relative grading (also called norm-referenced grading) ranks students against each other rather than against a fixed absolute scale. The top performers receive the highest grades regardless of the raw marks. This approach accounts for variation in exam difficulty and ensures a consistent grade distribution across different semesters.

3.2 GPA Scale Used

Grade	GPA Points	Rank Position	Students
A	4.0	Top 10	Positions 1–10
A-	3.7	11–15	Positions 11–15
B+	3.3	16–20	Positions 16–20
B	3.0	21–25	Positions 21–25
B-	2.7	26–30	Positions 26–30
C+	2.3	31–35	Positions 31–35
C	2.0	36–40	Positions 36–40
D	1.0	41–45	Positions 41–45
F	0.0	46+	All remaining

3.3 CGPA Calculation:

CGPA is the weighted average of GPA points across all enrolled papers, calculated as:

$$\text{CGPA} = \frac{\sum(\text{GPA} \times \text{Credit Hours})}{\sum(\text{Credit Hours})}$$

4. Program Description:

The program is a console-based Relative Grading System that allows a teacher or administrator to manage marks for multiple papers and students. The main functional areas are:

- Enter paper details including student marks
- Update existing student records
- View per-paper result sheets with topper
- Generate a DMC (Detailed Marks Certificate) for any student

4.1 Data Structures:

Structure / Array	Type	Purpose
Student	struct	Holds roll number and name
PaperRecord	struct	Holds student, paper name, credit hours, marks, grade, GPA
records[]	PaperRecord[500]	Global array storing all student-paper entries
paperNames[]	string[10]	Stores unique paper names entered so far
totalRecords	int	Count of all records stored
totalPapers	int	Count of unique papers registered

4.2 Functions Overview

Function	Description
paperExists()	Checks if a paper name already exists in paperNames[]
addPaper()	Adds a new paper name to the global list if not duplicate
duplicateRecord()	Checks for duplicate (rollNo, paper) combination
assignGrades()	Sorts students of one paper by marks and assigns relative grades
showTopper()	Finds and displays the top-marks student for a given paper
enterDetails()	Prompts for paper info and student marks, validates inputs
updateDetails()	Finds a record by roll no. and paper, allows update of name/marks/roll
paperDetailsMenu()	Sub-menu: Enter or Update paper details
seePaperResults()	Lists all papers, lets user pick one, displays formatted result table
showDMC()	Builds unique student list, shows full DMC with CGPA for selected student
main()	Entry point; runs do-while loop with 4-option main menu

5. Code Walkthrough

5.1 Structures

Two nested structures are defined:

```
struct Student {
```

```

    string rollNo;
    string name;
};

struct PaperRecord {
    Student student;    // nested struct
    string paperName;
    int    creditHours;
    float  marks;
    string grade;
    float  gpa;
};

```

5.2 Relative Grade Assignment (assignGrades)

The function collects indexes of all records for a given paper, sorts them in descending order of marks using a simple selection sort on the index array, then assigns grades according to rank position:

```

// Sort indexes descending by marks
for (int i = 0; i < count - 1; i++)
    for (int j = i+1; j < count; j++)
        if (records[indexes[j]].marks > records[indexes[i]].marks)
            swap(indexes[i], indexes[j]);

// Assign grades by rank
if (i < 10)    grade = "A",    gpa = 4.0;
else if (i < 15) grade = "A-",  gpa = 3.7;
else if (i < 20) grade = "B+",  gpa = 3.3;
// ... and so on

```

5.3 Input Validation

```

while (temp.marks < 0 || temp.marks > 100) {
    cout << "Invalid Marks. Enter Again: ";
    cin  >> temp.marks;
}

```

5.4 CGPA Calculation (showDMC)

```

float totalPoints = 0;
int    totalCH    = 0;

for (int i = 0; i < totalRecords; i++) {
    if (records[i].student.rollNo == selectedRoll) {
        totalPoints += records[i].gpa * records[i].creditHours;
        totalCH     += records[i].creditHours;
    }
}
float cgpa = totalPoints / totalCH;

```

6. Program Flow

The program begins at main() and enters a do-while loop presenting the main menu. The control flow for each option is summarised below:

Option	Flow
1	paperDetailsMenu() → Enter Details: validate paper, loop through students, validate marks, store record, call assignGrades() OR Update Details: locate record, offer name/marks/roll update, re-grade if marks changed
2	seePaperResults(): list papers → user picks → print formatted table of all students in that paper → showTopper()

3	showDMC(): build unique student list → user picks → print all papers for that student with CGPA
4	Exit do-while loop, program ends

7. Sample Output

7.1 Main Menu

```

=====
      RELATIVE GRADING SYSTEM
=====
1. Enter Paper Details
2. See Each Paper Results
3. See DMC Transcript
4. Exit
Enter Choice:

```

7.2 Paper Result Table

```

===== Mathematics RESULT =====
Roll No      Name      Marks   Grade   GPA
2023-CS-10   Ali       92      A       4.0
2023-CS-05   Sara      88      A       4.0
2023-CS-17   Ahmed     85      A       4.0
...

```

```

===== TOPPER =====
Paper : Mathematics
Name  : Ali
Marks : 92

```

7.3 DMC Transcript

```

===== DMC =====
Name      : Ali
Roll No   : 2023-CS-10

Subject   CH      Marks   Grade   GPA
Mathematics 3       92      A       4.0
Physics    3       78      B+      3.3
English    2       70      B       3.0

Total Credit Hours: 8
CGPA: 3.54

```

8. Analysis & Discussion

8.1 Strengths

- Nested structures cleanly separate student identity from paper performance data.
- Relative grading is re-evaluated every time a record is added or marks are updated, ensuring consistency.
- Duplicate detection prevents the same student being entered twice for the same paper.
- Input validation ensures marks remain within the 0–100 range.
- showDMC() dynamically aggregates data without storing CGPA separately, reducing redundancy.

8.2 Limitations

- Data is not persisted to a file; all records are lost when the program exits.
- Student and paper name fields do not support spaces (cin stops at whitespace).

- The fixed arrays (MAX_RECORDS = 500, MAX_PAPERS = 10) impose hard limits.
- The grade bands are hardcoded for exactly 45+ students; smaller classes may result in very few students receiving failing grades or an unbalanced distribution.

8.3 Suggested Improvements

- Add file I/O (fstream) to save and load records between sessions.
- Use getline() for names to support multi-word inputs.
- Replace raw arrays with vectors for dynamic sizing.
- Implement a search/filter function by student name or roll number.
- Add an absolute minimum pass mark (e.g. 40%) before applying relative bands.

9. Conclusion

This lab project successfully implements a console-based Relative Grading System in C++. It demonstrates key programming concepts including nested structures, 1D arrays, sorting algorithms, modular function design, input validation, and formatted output with setw(). The system correctly computes per-paper grades and per-student CGPA, and provides a working DMC generation feature. The project serves as a practical application of C++ fundamentals to a real-world academic problem.

References

1. Bjarne Stroustrup, The C++ Programming Language, 4th Edition, Addison-Wesley, 2013.
2. Stanley B. Lippman, C++ Primer, 5th Edition, Addison-Wesley, 2012.
3. cppreference.com — C++ Standard Library Reference. [http](http://cppreference.com)